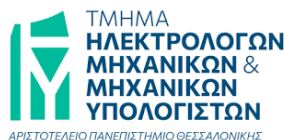


Αριστοτέλειο Πανεπιστήμιο Θεσσαλονίκης
Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών



Γραφική με Υπολογιστές

Εργασία 1: Μετασχηματισμοί και Προβολές

Τζίνα Θεοδώρα
10715
tzinatheod@ece.auth.gr

Εαρινό Εξάμηνο 2025-2026

Περιεχόμενα

1	Εισαγωγή	3
2	Μετασχηματισμοί Affine	4
2.1	Θεωρητική Ανάλυση	4
2.1.1	Ορισμός και Γενική Μορφή	4
2.1.2	Μετατόπιση (Translation)	4
2.1.3	Περιστροφή γύρω από τυχαίο άξονα (Rodrigues' Rotation Formula)	4
2.1.4	Σύνθεση Μετασχηματισμών και Κέντρο Περιστροφής	5
2.1.5	Αλλαγή Συστήματος Συντεταγμένων	5
2.2	Υλοποίηση σε Python	5
3	Μοντέλο Κάμερας, Προοπτική Προβολή και Απεικόνιση	7
3.1	Θεωρητική Ανάλυση	7
3.1.1	Προσδιορισμός Συστήματος Κάμερας (Look-at)	7
3.1.2	Μοντέλο Pinhole και Προοπτική Προβολή	7
3.1.3	Απεικόνιση στο Χώρο της Εικόνας (Rasterization)	8
3.2	Υλοποίηση σε Python	8
4	Διαδικασία Φωτογράφισης (Rendering Pipeline)	10
4.1	Θεωρητική Ανάλυση	10
4.2	Υλοποίηση σε Python	11
5	Scripts Επίδειξης (Demos)	12
5.1	Προετοιμασία Δεδομένων	12
5.2	Demo 1: Περιστροφή Αντικειμένου	12
5.3	Demo 2: Περιστροφή Κάμερας	13
5.4	Αποτελέσματα και Συμπεράσματα	14
5.4.1	Αποτελέσματα Demo 1: Περιστροφή Αντικειμένου	15
5.4.2	Αποτελέσματα Demo 2: Κυκλική Τροχιά Κάμερας	16

1 Εισαγωγή

Αντικείμενο της παρούσας εργασίας αποτελεί η μελέτη και η υλοποίηση του pipeline απεικόνισης τρισδιάστατων αντικειμένων σε δισδιάστατο καμβά, μέσω του μοντέλου της pinhole κάμερας και των affine μετασχηματισμών. Η διαδικασία αυτή αποτελεί τη συνέχεια της προηγούμενης εργασίας, επεκτείνοντας τον χρωματισμό τριγώνων με την έννοια της προοπτικής προβολής και της κίνησης στον τρισδιάστατο χώρο.

Η εργασία αναπτύχθηκε σε περιβάλλον Python, χρησιμοποιώντας τη βιβλιοθήκη NumPy για τη βελτιστοποίηση των υπολογισμών πινάκων και τη Matplotlib για την παραγωγή και αποθήκευση των τελικών εικόνων. Η υλοποίηση καλύπτει τα εξής βασικά στάδια:

- **Affine Μετασχηματισμοί:** Υλοποίηση της κλάσης `Trafo` για την εφαρμογή περιστροφής και μετατόπισης σε αντικείμενα του WCS.
- **Προσανατολισμός Κάμερας:** Καθορισμός του συστήματος συντεταγμένων της κάμερας (CCS) με βάση το κέντρο της κάμερας, το σημείο στόχο και το up διάνυσμα.
- **Προοπτική Προβολή:** Μετατροπή των 3D συντεταγμένων του κόσμου σε 2D συντεταγμένες στο επίπεδο της κάμερας, λαμβάνοντας υπόψη την εστιακή απόσταση και το βάθος.
- **Απεικόνιση (Rasterization):** Αντιστοίχιση των συνεχών συντεταγμένων του επιπέδου προβολής σε ακέραιες θέσεις pixels του καμβά.

Για την παραγωγή των τελικών αποτελεσμάτων, χρησιμοποιήθηκαν δύο σενάρια προσομοίωσης (demos) διάρκειας 1 δευτερολέπτου με ρυθμό 25 fps, αναδεικνύοντας τη λειτουργία της κάμερας και των μετασχηματισμών σε δυναμικό περιβάλλον.

2 Μετασχηματισμοί Affine

Στην ενότητα αυτή αναλύεται το θεωρητικό υπόβαθρο των affine μετασχηματισμών και η υλοποίησή τους μέσω της κλάσης **Trafo**. Οι μετασχηματισμοί αυτοί είναι θεμελιώδεις για τη Γραφική με Υπολογιστές, καθώς επιτρέπουν την τροποποίηση της θέσης και του προσανατολισμού των αντικειμένων διατηρώντας την παραλληλία των γραμμών.

2.1 Θεωρητική Ανάλυση

2.1.1 Ορισμός και Γενική Μορφή

Ένας affine μετασχηματισμός $\mathcal{T}$ στον τρισδιάστατο χώρο ορίζεται ως η απεικόνιση που συνδυάζει έναν γραμμικό μετασχηματισμό (περιστροφή, κλιμάκωση) και μια μετατόπιση. Μαθηματικά, για ένα σημείο $\mathbf{p} = [x, y, z]^T$, η μετασχηματισμένη θέση $\mathbf{p}'$ δίνεται από τη σχέση:

$$\mathbf{p}' = \mathbf{R}\mathbf{p} + \mathbf{t} \quad (1)$$

όπου ο πίνακας $\mathbf{R} \in \mathbb{R}^{3 \times 3}$ περιγράφει την περιστροφή και το διάνυσμα $\mathbf{t} \in \mathbb{R}^3$ τη μετατόπιση. Παρόλο που οι μετασχηματισμοί αυτοί μπορούν να αναπαρασταθούν με πίνακες 4×4 σε ομογενείς συντεταγμένες, στην παρούσα εργασία χρησιμοποιείται η μη ομογενής μορφή, διαχωρίζοντας τον πίνακα περιστροφής και το διάνυσμα μετατόπισης.

2.1.2 Μετατόπιση (Translation)

Η μετατόπιση είναι ο απλούστερος affine μετασχηματισμός, όπου κάθε σημείο μετακινείται κατά μια σταθερή απόσταση προς μια καθορισμένη κατεύθυνση $\mathbf{d} = [d_x, d_y, d_z]^T$:

$$\mathbf{p}_{new} = \mathbf{p}_{old} + \mathbf{d} \quad (2)$$

Στην υλοποίησή μας, η μετατόπιση είναι σωρευτική, δηλαδή προστίθεται στο υπάρχον διάνυσμα μετατόπισης του αντικειμένου.

2.1.3 Περιστροφή γύρω από τυχαίο άξονα (Rodrigues' Rotation Formula)

Για την περιστροφή γύρω από έναν άξονα που ορίζεται από το μοναδιαίο διάνυσμα $\mathbf{n}$ κατά γωνία θ , χρησιμοποιούμε τον τύπο του Rodrigues, ο οποίος επιτρέπει τον υπολογισμό του πίνακα $\mathbf{R}$ χωρίς τη χρήση Euler γωνιών:

$$\mathbf{R}(\mathbf{n}, \theta) = \mathbf{I} \cos \theta + (1 - \cos \theta) \mathbf{n}\mathbf{n}^T + \mathbf{K} \sin \theta \quad (3)$$

όπου $\mathbf{I}$ ο μοναδιαίος πίνακας και $\mathbf{K}$ ο αντισυμμετρικός πίνακας (skew-symmetric matrix) του $\mathbf{n}$. Ο πίνακας $\mathbf{K}$ αναπαριστά τον τελεστή του εξωτερικού γινομένου, τέτοιον ώστε για κάθε διάνυσμα $\mathbf{p}$ να ισχύει $\mathbf{K}\mathbf{p} = \mathbf{n} \times \mathbf{p}$. Ορίζεται ως:

$$\mathbf{K} = \begin{bmatrix} 0 & -n_z & n_y \\ n_z & 0 & -n_x \\ -n_y & n_x & 0 \end{bmatrix} \quad (4)$$

2.1.4 Σύνθεση Μετασχηματισμών και Κέντρο Περιστροφής

Η περιστροφή ενός αντικειμένου γύρω από ένα τυχαίο σημείο $\mathbf{c}$ (center) απαιτεί τη μεταφορά του σημείου στην αρχή των αξόνων, την περιστροφή και την επαναφορά του. Αυτό οδηγεί σε μια νέα κατάσταση του αντικειμένου όπου ο πίνακας περιστροφής και το διάνυσμα μετατόπισης ενημερώνονται ως εξής:

- $\mathbf{R}_{new} = \mathbf{R}_{rot} \cdot \mathbf{R}_{old}$
- $\mathbf{t}_{new} = \mathbf{R}_{rot} \cdot \mathbf{t}_{old} + \mathbf{c} - \mathbf{R}_{rot} \cdot \mathbf{c}$

2.1.5 Αλλαγή Συστήματος Συντεταγμένων

Η μετάβαση σημείων από ένα αυθαίρετο σύστημα συντεταγμένων σε ένα άλλο απαιτεί τον μετασχηματισμό τους πρώτα στο Παγκόσμιο Σύστημα Συντεταγμένων (WCS) και έπειτα στο σύστημα-στόχο. Αν $\mathbf{R}_{src}$ είναι ο πίνακας του αρχικού συστήματος, ένα σημείο $\mathbf{p}_{src}$ μεταφέρεται στο WCS ως $\mathbf{p}_{world} = \mathbf{R}_{src} \cdot \mathbf{p}_{src}$. Η προβολή του $\mathbf{p}_{world}$ στο τοπικό σύστημα ενός αντικειμένου (που ορίζεται από $\mathbf{R}_{tgt}$ και $\mathbf{t}_{tgt}$) υπολογίζεται μέσω του αντίστροφου μετασχηματισμού:

$$\mathbf{p}_{target} = \mathbf{R}_{tgt}^{-1}(\mathbf{p}_{world} - \mathbf{t}_{tgt}) = \mathbf{R}_{tgt}^T(\mathbf{p}_{world} - \mathbf{t}_{tgt}) \quad (5)$$

Η ιδιότητα $\mathbf{R}^{-1} = \mathbf{R}^T$ ισχύει διότι οι πίνακες περιστροφής είναι ορθογώνιοι.

2.2 Υλοποίηση σε Python

Η υλοποίηση των affine μετασχηματισμών πραγματοποιήθηκε στο αρχείο `transforms.py` μέσω της κλάσης `Trafo`. Η κλάση αυτή διατηρεί την τρέχουσα γεωμετρική κατάσταση του αντικειμένου χρησιμοποιώντας έναν πίνακα περιστροφής 3×3 και ένα διάνυσμα μετατόπισης 3×1 .

Listing 1: Σωρευτική δεξιόστροφη περιστροφή και διαχείριση κέντρου

```
# Negate angle to produce clockwise rotation (right-hand rule)
c, s = np.cos(-angle), np.sin(-angle)

# Rodrigues' formula for rotation matrix R
R = c * np.eye(3) + (1 - c) * np.outer(n, n) + s * K

# Update rotation matrix: new_rot = R_new @ old_rot
self.rot_mat = R @ self.rot_mat

# Update translation: new_t = R_new @ old_t + center - R_new @ center
self.t_vec = R @ self.t_vec + center - R @ center
```

- **Διατήρηση Κατάστασης:** Οι μέθοδοι `rotate` και `translate` υλοποιήθηκαν έτσι ώστε να ενημερώνουν εσωτερικά τα μέλη της κλάσης (`self.rot_mat`, `self.t_vec`). Αυτό επιτρέπει τη σύνθεση πολλαπλών μετασχηματισμών με διαδοχικές κλήσεις, χωρίς ο χρήστης να χρειάζεται να διαχειρίζεται ενδιάμεσους πίνακες.

- **Σύμβαση Κέντρου Περιστροφής:** Η υλοποίηση της περιστροφής επιτρέπει τον ορισμό ενός τυχαίου σημείου `center` ως κέντρου. Όπως φαίνεται στο Listing 1, η μετατόπιση `t_vec` προσαρμόζεται δυναμικά ώστε να αντικατοπτρίζει τη γεωμετρική διόρθωση που απαιτείται όταν ο άξονας δεν διέρχεται από την αρχή των αξόνων.
- **Κανονικοποίηση Άξονα:** Πριν την εφαρμογή του τύπου Rodrigues, ο άξονας περιστροφής κανονικοποιείται (`unit vector`) μέσω της `np.linalg.norm`, εξασφαλίζοντας την ορθότητα του πίνακα $\mathbf{R}$ ανεξάρτητα από το μέτρο του διανύσματος εισόδου.
- **Δεξιόστροφη Περιστροφή:** Σε ένα σύστημα συντεταγμένων όπου ο άξονας +Y δείχνει προς τα πάνω, ο κανόνας του δεξιού χεριού (Right-Hand Rule) παράγει από προεπιλογή αριστερόστροφη κίνηση για θετικές γωνίες. Δεδομένου ότι η εκφώνηση απαιτεί η μέθοδος να εφαρμόζει δεξιόστροφη περιστροφή, η γωνία `angle` αρνητικοποιείται κατά τον υπολογισμό των τριγωνομετρικών παραμέτρων (`c`, `s`).

Listing 2: Μετασχηματισμός σημείων και αλλαγή βάσης

```
# Application of the full affine transform p' = Rp + t
transformed_pts = self.rot_mat @ pts + self.t_vec[:, np.newaxis]

# System to System conversion (sys_src -> WCS -> target)
p_world = sys_src @ pts
p_target = self.rot_mat.T @ (p_world - self.t_vec[:, np.newaxis])
```

- **Βελτιστοποιημένη Επεξεργασία (Vectorization):** Στη μέθοδο `xform_pnts`, η εφαρμογή του μετασχηματισμού σε N σημεία γίνεται μαζικά. Η χρήση του `np.newaxis` για το διάνυσμα μετατόπισης επιτρέπει στη NumPy να εφαρμόσει *broadcasting*, αποφεύγοντας τη χρήση βρόχων (`loops`) που θα μείωναν την απόδοση.
- **Αντιστροφή μέσω Αναστροφής:** Στη μέθοδο `sys2sys`, η μετάβαση από το WCS στο τοπικό σύστημα απαιτεί την αντιστροφή του μετασχηματισμού. Χρησιμοποιείται η ανάστροφη του πίνακα (`.T`), εκμεταλλευόμενοι το γεγονός ότι οι πίνακες περιστροφής είναι ορθογώνιοι ($\mathbf{R}^{-1} = \mathbf{R}^T$), γεγονός που ελαχιστοποιεί το υπολογιστικό κόστος.
- **Μη Ομογενής Αναπαράσταση:** Παρόλο που η εκφώνηση αναφέρει τη δυνατότητα χρήσης 4×4 πίνακα, επιλέχθηκε ο διαχωρισμός σε R (3×3) και t (3×1). Η επιλογή αυτή έγινε για να υπάρχει απόλυτη συμβατότητα με τις συναρτήσεις `lookat` και `perspective_project` που απαιτούν ορίσματα σε αυτή τη μορφή.

3 Μοντέλο Κάμερας, Προοπτική Προβολή και Απεικόνιση

Σε αυτή την ενότητα αναλύεται ο καθορισμός της θέσης και του προσανατολισμού της κάμερας στον τρισδιάστατο χώρο, η μαθηματική διαδικασία της προοπτικής προβολής των σημείων στο επίπεδο της εικόνας, καθώς και η μετατροπή των σημείων από το συνεχές επίπεδο προβολής στο διακριτό πλέγμα των pixels (rasterization).

3.1 Θεωρητική Ανάλυση

3.1.1 Προσδιορισμός Συστήματος Κάμερας (Look-at)

Η μετάβαση από το Παγκόσμιο Σύστημα (WCS) στο Σύστημα Συντεταγμένων Κάμερας (CCS) απαιτεί τον προσδιορισμό μιας ορθοκανονικής βάσης στον χώρο. Το CCS ορίζεται από το κέντρο της κάμερας $\mathbf{e}$ (eye), ένα σημείο στόχο $\mathbf{a}$ (target) και ένα διάνυσμα $\mathbf{v}_{up}$ που ορίζει την κατεύθυνση "πάνω". Οι άξονες του συστήματος υπολογίζονται ως εξής:

1. **Άξονας $\mathbf{z}_{cam}$:** Καθορίζεται από τη διεύθυνση θέασης. Επειδή η κάμερα κοιτάζει προς τον αρνητικό άξονα Z , το διάνυσμα $\mathbf{z}_{cam}$ δείχνει από το στόχο προς την κάμερα:

$$\mathbf{z}_{cam} = \frac{\mathbf{e} - \mathbf{a}}{|\mathbf{e} - \mathbf{a}|} \quad (6)$$

2. **Άξονας $\mathbf{x}_{cam}$:** Είναι κάθετος στο επίπεδο που ορίζουν το $\mathbf{v}_{up}$ και ο $\mathbf{z}_{cam}$ (δεξιόστροφο σύστημα):

$$\mathbf{x}_{cam} = \frac{\mathbf{v}_{up} \times \mathbf{z}_{cam}}{|\mathbf{v}_{up} \times \mathbf{z}_{cam}|} \quad (7)$$

3. **Άξονας $\mathbf{y}_{cam}$:** Επαναυπολογίζεται για να διασφαλιστεί η ορθογωνιότητα: $\mathbf{y}_{cam} = \mathbf{z}_{cam} \times \mathbf{x}_{cam}$

Ο πίνακας περιστροφής $\mathbf{R}$ έχει ως γραμμές τα παραπάνω διανύσματα, ενώ το διάνυσμα μετατόπισης $\mathbf{t}$ μεταφέρει την αρχή των αξόνων στο $\mathbf{e}$: $\mathbf{t} = -\mathbf{R}\mathbf{e}$.

3.1.2 Μοντέλο Pinhole και Προοπτική Προβολή

Το μοντέλο της κάμερας οπής (pinhole camera) προσομοιώνει την προβολή σημείων $\mathbf{p}_{ccs} = [x_c, y_c, z_c]^T$ σε ένα επίπεδο (camera plane) σε απόσταση f (focal length) από το κέντρο προβολής. Λόγω ομοιότητας τριγώνων, οι προβαλλόμενες συντεταγμένες (x_p, y_p) δίνονται από τους τύπους:

$$x_p = f \cdot \frac{x_c}{d}, \quad y_p = f \cdot \frac{y_c}{d} \quad (8)$$

όπου $d = -z_c$ αντιπροσωπεύει το θετικό βάθος (depth) του σημείου από το επίπεδο της κάμερας.

3.1.3 Απεικόνιση στο Χώρο της Εικόνας (Rasterization)

Μετά την προβολή, οι συνεχείς συντεταγμένες (x_p, y_p) ανήκουν στο επίπεδο της κάμερας, το οποίο έχει διαστάσεις $W_p \times H_p$ (plane_w, plane_h) και κέντρο το $(0, 0)$. Αυτές πρέπει να αντιστοιχηθούν σε ακέραιες θέσεις pixels (P_x, P_y) μιας εικόνας ανάλυσης $res_w \times res_h$. Η διαδικασία περιλαμβάνει τη μετατόπιση της αρχής των αξόνων στην κάτω αριστερή γωνία (προσθέτοντας $W_p/2$ και $H_p/2$ αντίστοιχα) και την κατάλληλη κλιμάκωση:

$$P_x = \text{round} \left(\frac{x_p + W_p/2}{W_p} \cdot res_w \right), \quad P_y = \text{round} \left(\frac{y_p + H_p/2}{H_p} \cdot res_h \right) \quad (9)$$

3.2 Υλοποίηση σε Python

Η υλοποίηση συγκεντρώθηκε στο αρχείο `camera.py`, διαχωρίζοντας τη λογική της κάμερας από τους γενικούς μετασχηματισμούς αντικειμένων.

Listing 3: Υπολογισμός παραμέτρων προσανατολισμού (lookat)

```
# Z-axis points from target to eye
z_cam = (eye - target) / np.linalg.norm(eye - target)
# X-axis is perpendicular to up and z_cam
x_cam = np.cross(up, z_cam) / np.linalg.norm(np.cross(up, z_cam))
# Y-axis completes the orthonormal basis
y_cam = np.cross(z_cam, x_cam)

R = np.stack([x_cam, y_cam, z_cam], axis=0)
t = -R @ eye
```

- **Ορθοκανονικοποίηση:** Η χρήση εξωτερικών γινομένων (`np.cross`) και κανονικοποίησης (`np.linalg.norm`) διασφαλίζει ότι η βάση της κάμερας είναι ορθοκανονική, αποτρέποντας παραμορφώσεις στην τελική προβολή.
- **Σύμβαση Αξόνων:** Ακολουθήθηκε η σύμβαση όπου ο άξονας $-Z$ ταυτίζεται με τη διεύθυνση θέασης. Αυτό αντικατοπτρίζεται στον υπολογισμό του `z_cam`, ο οποίος ορίζεται με φορά προς τον παρατηρητή.
- **Μεταφορά στο CCS:** Η συνάρτηση επιστρέφει το ζεύγος (R, t) , το οποίο επιτρέπει τη μετατροπή οποιουδήποτε σημείου του κόσμου απευθείας στο σύστημα της κάμερας μέσω της πράξης $R \cdot p + t$.

Listing 4: Προοπτική προβολή και διαχείριση βάθους (perspective_project)

```
# Project vertices from CCS to the camera plane
depth = -p_cam[2] # Positive depth = -z_cam
x_2d = focal * p_cam[0] / depth
y_2d = focal * p_cam[1] / depth

pts_2d = np.stack([x_2d, y_2d], axis=0)
```


- **Διαίρεση Βάθους:** Η υλοποίηση εκμεταλλεύεται το *broadcasting* της NumPy για να εκτελέσει τη διαίρεση με το βάθος ταυτόχρονα για όλα τα σημεία εισόδου, βελτιώνοντας την ταχύτητα εκτέλεσης του pipeline.
- **Εξαγωγή Βάθους:** Εκτός από τις 2D συντεταγμένες, η συνάρτηση επιστρέφει και το διάνυσμα *depth*. Η πληροφορία αυτή είναι κρίσιμη για το στάδιο του rendering, καθώς επιτρέπει την ταξινόμηση των τριγώνων (Painter's Algorithm).
- **Ασφάλεια Υπολογισμών:** Στον κώδικα περιλαμβάνεται έλεγχος για σημεία με μηδενικό ή αρνητικό βάθος (πίσω από την κάμερα), ώστε να αποφεύγονται σφάλματα διαίρεσης με το μηδέν ή λανθασμένες προβολές "καθρεπτισμού".

Listing 5: Αντιστοίχιση σε Pixels (rasterize)

```
# Shift from [-plane_w/2, plane_w/2] to [0, plane_w], then scale
px = np.round((x_plane + plane_w / 2.0) / plane_w * res_w).astype(int)
py = np.round((y_plane + plane_h / 2.0) / plane_h * res_h).astype(int)
```

- **Ακριβής Δειγματοληψία:** Η χρήση της συνάρτησης `np.round` εξασφαλίζει ότι οι συνεχείς συντεταγμένες προβολής αντιστοιχίζονται ορθά στο πλησιέστερο διακριτό pixel, ελαχιστοποιώντας τα σφάλματα γεωμετρικής μετατόπισης.
- **Ακεραιοποίηση:** Η κλήση της `.astype(int)` είναι απολύτως απαραίτητη, καθώς οι δείκτες που επιστρέφονται θα χρησιμοποιηθούν αργότερα για την προσπέλαση του πίνακα εικόνας (image array indexing), η οποία απαιτεί αυστηρά ακέραιους τύπους.

4 Διαδικασία Φωτογράφισης (Rendering Pipeline)

Η διαδικασία της "φωτογράφισης" (rendering) αποτελεί τη σύνθεση όλων των προηγούμενων σταδίων σε μια ενιαία ροή εργασίας (pipeline). Στόχος είναι η μετατροπή των τρισδιάστατων δεδομένων του αντικειμένου από το Παγκόσμιο Σύστημα (WCS) σε μια έγχρωμη ψηφιακή εικόνα, αξιοποιώντας τις μεθόδους σκίασης που αναπτύχθηκαν στην προηγούμενη εργασία.

4.1 Θεωρητική Ανάλυση

Η ολοκληρωμένη διαδικασία απόδοσης ακολουθεί μια αυστηρή σειρά βημάτων, ώστε να διασφαλιστεί η γεωμετρική ορθότητα και η σωστή προοπτική:

1. **Προσδιορισμός View Matrix:** Υπολογισμός των παραμέτρων της κάμερας (πίνακας περιστροφής $\mathbf{R}$, διάνυσμα μετατόπισης $\mathbf{t}$) μέσω της συνάρτησης `lookat`. Η διαδικασία αυτή ορίζει τη μετάβαση από το Παγκόσμιο Σύστημα (WCS) στο Σύστημα Συντεταγμένων Κάμερας (CCS), λαμβάνοντας υπόψη τη θέση του παρατηρητή (`eye`), το σημείο εστίασης (`target`) και το διάνυσμα αναφοράς (`up`).
2. **Προβολή στο Επίπεδο:** Εφαρμογή προοπτικής προβολής μέσω της `perspective_project` για τη μετατροπή των 3D κορυφών από το CCS σε 2D συντεταγμένες στο επίπεδο της κάμερας. Η διαδικασία χρησιμοποιεί την εστιακή απόσταση (`focal`) και παράλληλα εξάγει το διάνυσμα βάθους (`depth`), το οποίο είναι απολύτως αναγκαίο για τη μετέπειτα επίλυση του προβλήματος των κορυφών επιφανειών.
3. **Ψηφιοποίηση (Rasterization):** Μετατροπή των συνεχών 2D συντεταγμένων του επιπέδου της κάμερας σε ακέραιες θέσεις (pixels) στον ψηφιακό καμβά, μέσω της συνάρτησης `rasterize`. Αυτό επιτυγχάνεται μέσω κατάλληλης μετατόπισης του κέντρου των αξόνων και κλιμάκωσης των τιμών βάσει των φυσικών διαστάσεων του πετάσματος (`plane_w`, `plane_h`) και της ανάλυσης της εικόνας (`res_w`, `res_h`).
4. **Διόρθωση Συστήματος Συντεταγμένων:** Προσαρμογή των κατακόρυφων συντεταγμένων (pixels) ώστε να ευθυγραμμίζονται με τη δομή αποθήκευσης της εικόνας στη μνήμη (πίνακες της NumPy). Επειδή η αρίθμηση των γραμμών (`rows`) σε έναν πίνακα ξεκινά από πάνω προς τα κάτω, ο άξονας Y του συστήματος raster αναστρέφεται γεωμετρικά μέσω του μετασχηματισμού $y_{img} = (res_h - 1) - y_{raster}$.
5. **Σκίαση και Πλήρωση:** Κλήση της συνάρτησης `render_img`, η οποία έχει υλοποιηθεί στο πλαίσιο της προηγούμενης εργασίας, για τον τελικό χρωματισμό των τριγώνων. Η ρουτίνα αυτή παραλαμβάνει τις τελικές 2D συντεταγμένες και το βάθος, εκτελεί ταξινόμηση των τριγώνων (Painter's Algorithm) και τα ζωγραφίζει χρησιμοποιώντας δυναμικά είτε γραμμική παρεμβολή (Gouraud) είτε δειγματοληψία εικόνας (Texture Mapping).

4.2 Υλοποίηση σε Python

Η κεντρική συνάρτηση `render_object` στο αρχείο `renderer.py` υλοποιεί τον παραπάνω αλγόριθμο, λειτουργώντας ως ενορχηστρωτής του pipeline.

Listing 6: Η ροή της συνάρτησης `render_object`

```
# Step 1: Compute camera extrinsic parameters
R, t = lookat(eye, up, target)

# Step 2: Project 3D vertices onto the camera plane
pts_world = v_pos.T
pts_2d, depth = perspective_project(pts_world, focal, R, t)

# Step 3: Rasterize to pixel coordinates (y=0 at bottom)
pix = rasterize(pts_2d, plane_w, plane_h, res_w, res_h)

# Step 4: Flip y so that row 0 is the top of the image array
py_img = (res_h - 1) - pix[1]
vertices_2d = np.stack([pix[0], py_img], axis=1).astype(int)

# Step 5: Final rendering with shading selection
img = render_img(faces=t_pos_idx, vertices=vertices_2d, vcolors=v_clr,
                 uvs=uvvs, depth=depth, shading=shading, texImg=texImg)
```

- **Διαχείριση Διαστάσεων:** Στο Βήμα 2, ο πίνακας `v_pos` μετατρέπεται σε transposed (`.T`) για να αποκτήσει μορφή $(3, N)$, όπως απαιτείται για τις πράξεις πινάκων, καθώς η αρχική του μορφή είναι $(N, 3)$.
- **Συμβατότητα με Προηγούμενη Εργασία:** Στο Βήμα 4, οι συντεταγμένες x και y συνενώνονται (`np.stack`) σε έναν πίνακα $(N, 2)$ ακεραίων. Αυτή η μορφή επιτρέπει την άμεση επαναχρησιμοποίηση της συνάρτησης `render_img` από την πρώτη εργασία.
- **Επιλογή Σκίασης:** Η υλοποίηση ελέγχει δυναμικά την ύπαρξη δεδομένων υφής. Εάν παρέχονται τα `v_uvs` και `texImg`, ενεργοποιείται η μέθοδος Texture Mapping (`'t'`), διαφορετικά το σύστημα εκτελεί Gouraud Shading (`'g'`).
- **Επίλυση Ορατότητας:** Η πληροφορία `depth` που παρήχθη κατά την προβολή μεταβιβάζεται στη `render_img`, η οποία χρησιμοποιεί τον αλγόριθμο του ζωγράφου (Painter's Algorithm) για τη σωστή επικάλυψη των τριγώνων βάσει απόστασης.

5 Scripts Επίδειξης (Demos)

Για την επαλήθευση της ορθής λειτουργίας των μετασχηματισμών και της κάμερας, δημιουργήθηκαν δύο αυτόνομα scripts (`demo1.py` και `demo2.py`). Και τα δύο παράγουν μια προσομοίωση (animation) διάρκειας 1 δευτερολέπτου στα 25 fps, εξάγοντας συνολικά 25 καρέ (frames) το καθένα.

5.1 Προετοιμασία Δεδομένων

Στην αρχή κάθε script επίδειξης, πραγματοποιείται η απαραίτητη φόρτωση και μορφοποίηση των δεδομένων της σκηνής από το λεξικό του αρχείου `data.npy`. Η διαδικασία αυτή οργανώνεται στις εξής βασικές κατηγορίες:

- **Γεωμετρία και Τοπολογία:** Αντλούνται οι 3D συντεταγμένες των κορυφών (`v_pos`), οι αντίστοιχες συντεταγμένες υψής (`v_uns`) και οι δείκτες που ορίζουν τη συνδεσμολογία των τριγώνων (`t_pos_idx`). Στο σημείο αυτό εφαρμόζεται μια σημαντική βελτιστοποίηση ως προς τη διαχείριση των διαστάσεων του `v_pos`:
 - Στο **Demo 1**, ο πίνακας φορτώνεται διατηρώντας την αρχική του διάσταση $(3, N)$. Αυτό γίνεται διότι το αντικείμενο πρέπει να περιστρέφεται συνεχώς. Η μέθοδος `xform_pnts` της κλάσης `Trafo` εκτελεί πολλαπλασιασμούς πινάκων και απαιτεί τα δεδομένα σε διανύσματα στήλης $(3, N)$. Διατηρώντας αυτή τη μορφή, αποφεύγουμε περιττές μετατροπές μέσα στον βρόχο. Η μετατροπή στην απαιτούμενη μορφή $(N, 3)$ για τη ρουτίνα `render_object` γίνεται στοχευμένα στο τέλος κάθε καρέ.
 - Αντίθετα, στο **Demo 2**, το αντικείμενο παραμένει ακίνητο. Επομένως, ο πίνακας αναστρέφεται απευθείας κατά τη φόρτωση (`v_pos.T`) λαμβάνοντας τη μορφή $(N, 3)$, αφού προορίζεται αποκλειστικά για την τροφοδότηση της συνάρτησης σχεδίασης χωρίς να μεσολαβούν άλλοι μετασχηματισμοί.
- **Παράμετροι Κάμερας και Animation:** Ανάλογα με το σενάριο, διαβάζονται οι εγγενείς παράμετροι του μοντέλου `pinhole`, όπως η εστιακή απόσταση (`focal`) και οι φυσικές διαστάσεις του πετάσματος (`plane_w`, `plane_h`). Παράλληλα, υπολογίζεται ο συνολικός αριθμός των παραγόμενων καρέ (`n_frames`) βάσει του ρυθμού ανανέωσης (`fps`) και της επιθυμητής διάρκειας του animation σε δευτερόλεπτα.
- **Επεξεργασία Υφής (Texture):** Η εικόνα `loony-repeat.png` φορτώνεται στη μνήμη και μετατρέπεται σε πίνακα RGB. Τα χρωματικά της δεδομένα κανονικοποιούνται στο διάστημα $[0, 1]$ (διαιρώντας με το 255.0). Τέλος, η εικόνα αναστρέφεται κατακόρυφα μέσω της συνάρτησης `np.flipud`, προκειμένου η χαρτογράφησης της πάνω στο 3D μοντέλο να ευθυγραμμίζεται σωστά με τις UV συντεταγμένες.

5.2 Demo 1: Περιστροφή Αντικειμένου

Στο πρώτο σενάριο (`demo1.py`), η κάμερα παραμένει σταθερή στη θέση που ορίζεται από το `k_cam_eye`, κοιτώντας προς το `k_cam_target`. Το αντικείμενο βρίσκεται στην αρχή των

αξόνων και εκτελεί μια πλήρη δεξιόστροφη περιστροφή γύρω από τον άξονα Y του WCS.

Η γωνία περιστροφής για κάθε καρέ i υπολογίζεται ως $\theta = 2\pi \cdot \frac{i}{N_{frames}}$. Για τον υπολογισμό της νέας θέσης των κορυφών δημιουργείται ένα στιγμιότυπο της κλάσης `Trafo`, το οποίο εφαρμόζει τον μετασχηματισμό στις αρχικές συντεταγμένες.

Listing 8: Υπολογισμός Κίνησης Αντικειμένου (Demo 1)

```
for frame in range(n_frames):
    # Positive angle is passed to rotate(), which internally applies CW rotation
    angle = 2.0 * np.pi * frame / n_frames

    tr = Trafo()
    tr.rotate(y_axis, angle, rot_center)

    # Rotate vertices: xform_pnts returns (3, N) -> transpose to (N, 3)
    v_pos_rot = tr.xform_pnts(v_pos).T

    # Render using v_pos_rot ...
```

- **Δεξιόστροφη Περιστροφή:** Η γωνία που περνάει στη μέθοδο `rotate` της κλάσης `Trafo` είναι θετική, όπου πραγματοποιείται εσωτερικά η μετατροπή της σε δεξιόστροφη κίνηση.
- **Αποφυγή Συσσώρευσης Σφαλμάτων:** Στον βρόχο (loop) του animation, το αντικείμενο `Trafo` αρχικοποιείται από την αρχή (`tr = Trafo()`) και ο μετασχηματισμός εφαρμόζεται πάντα στον αρχικό πίνακα `v_pos`. Αυτό αποτρέπει τη συσσώρευση σφαλμάτων κινητής υποδιαστολής (floating-point errors) που θα προέκυπταν αν εφαρμόζαμε την περιστροφή επανειλημμένα στο ήδη περιστραμμένο αντικείμενο του προηγούμενου καρέ.
- **Διαχείριση Διαστάσεων:** Όπως αναφέρθηκε στην προετοιμασία δεδομένων, η `xform_pnts` επιστρέφει τα σημεία σε μορφή $(3, N)$. Η αναστροφή (`.T`) εφαρμόζεται ακριβώς πριν την κλήση της `render_object`, διασφαλίζοντας τη συμβατότητα με τις προδιαγραφές της συνάρτησης $((N, 3))$.
- **Δυναμική Δέσμευση Χρωμάτων:** Κατά την κλήση της `render_object`, ο πίνακας χρωμάτων αρχικοποιείται δυναμικά ως `np.zeros((v_pos_rot.shape[0], 3))`, εξασφαλίζοντας ότι το πλήθος των χρωμάτων ταιριάζει απόλυτα με το πλήθος των κορυφών N του περιστραμμένου μοντέλου.

5.3 Demo 2: Περιστροφή Κάμερας

Στο δεύτερο σενάριο (`demo2.py`), το αντικείμενο παραμένει απόλυτα σταθερό στην αρχή των αξόνων. Αντίθετα, η κάμερα διαγράφει μια πλήρη κυκλική τροχιά γύρω από αυτό (περιστρέφεται γύρω από τον άξονα Y), διατηρώντας σταθερό ύψος (`eye_height`) και σταθερή ακτίνα (`radius`).

Η θέση της κάμερας **e** (eye) ενημερώνεται δυναμικά σε κάθε καρέ χρησιμοποιώντας παραμετρικές εξισώσεις, ενώ ο στόχος (**target**) παραμένει το κέντρο του συστήματος $[0, 0, 0]$.

Listing 9: Υπολογισμός Τροχιάς Κάμερας (Demo 2)

```
for frame in range(n_frames):
    # Negative angle for clockwise rotation (right-hand rule)
    angle = - 2.0 * np.pi * frame / n_frames

    # Camera position on the orbit circle
    eye = np.array([
        radius * np.sin(angle), # X coordinate
        eye_height,             # Y remains constant
        radius * np.cos(angle)  # Z coordinate
    ])

    # Render using the static v_pos and the new eye position ...
```

- **Δεξιόστροφη Παραμετρική Τροχιά:** Ο υπολογισμός των συντεταγμένων X και Z μέσω των τριγωνομετρικών συναρτήσεων $\sin(\theta)$ και $\cos(\theta)$ αντιστοίχως, εγγυάται τη διαγραφή μιας τέλειας κυκλικής τροχιάς ακτίνας r . Προκειμένου η κάμερα να κινηθεί δεξιόστροφα (clockwise) στο επίπεδο XZ σύμφωνα με τον κανόνα του δεξιού χεριού, η γωνία θ ορίζεται σκόπιμα με αρνητικό πρόσημο. Η κατακόρυφη συντεταγμένη Y διατηρείται σταθερή, προσομοιώνοντας μια ομαλή πλευρική λήψη του αντικείμενου.
- **Συνεχής Εστίαση:** Παρόλο που το κέντρο προβολής (**eye**) μεταβάλλεται συνεχώς, το διάνυσμα του στόχου (**target**) παραμένει αυστηρά $[0, 0, 0]$. Αυτό αναγκάζει το Σύστημα Κάμερας (CCS) να επαναυπολογίζει τους άξονές του σε κάθε καρέ (μέσω της **lookat**), διατηρώντας το αντικείμενο πάντα στο κέντρο του οπτικού πεδίου.
- **Στατική Γεωμετρία:** Επειδή το αντικείμενο είναι ακίνητο, ο πίνακας **v_pos** περνάει απευθείας στη συνάρτηση σχεδίασης χωρίς να υφίσταται κανέναν **affine** μετασχηματισμό στο Παγκόσμιο Σύστημα. Η αίσθηση της κίνησης παράγεται αποκλειστικά από την ενημέρωση του View Matrix (**R**, **t**) στο εσωτερικό της **render_object**.

5.4 Αποτελέσματα και Συμπεράσματα

Στο τελικό αυτό στάδιο της αναφοράς, παρουσιάζονται τα οπτικά αποτελέσματα (rendered frames) των δύο σεναρίων προσομοίωσης, με στόχο την πρακτική επαλήθευση της ορθότητας ολόκληρου του γραφικού pipeline. Κάθε προσομοίωση διαρκεί ακριβώς 1 δευτερόλεπτο και αποδίδεται στα 25 καρέ ανά δευτερόλεπτο (fps), παράγοντας συνολικά 25 στιγμιότυπα. Κατά την εκτέλεση των scripts, τα δεδομένα εξάγονται και αποθηκεύονται αυτόματα στους αντίστοιχους φακέλους **demo1_frames/** και **demo2_frames/**.

Μέσα από την οπτική ανάλυση αυτών των καρέ, αξιολογείται η ευστάθεια των μαθηματικών μετασχηματισμών, η ορθή εφαρμογή της προοπτικής προβολής, η σωστή λειτουργία του

αλγορίθμου ορατότητας (Painter's Algorithm), καθώς και η ακρίβεια της χαρτογράφησης υφής (Texture Mapping).

5.4.1 Αποτελέσματα Demo 1: Περιστροφή Αντικειμένου

Στο παρακάτω σχήμα παρατίθεται το σύνολο των 25 καρέ της πρώτης προσομοίωσης σε διάταξη πλέγματος (filmstrip), ώστε να γίνει αντιληπτή η αλληλουχία της κίνησης.



Figure 1: Σύνολο 25 καρέ από το Demo 1: Δεξιόστροφη περιστροφή του αντικειμένου γύρω από τον άξονα Y, με σταθερή κάμερα.

Για καλύτερη παρατήρηση της ροής κίνησης και της ποιότητας απόδοσης (rendering), το παρακάτω σχήμα απομονώνει 3 συνεχόμενα καρέ (11, 12 και 13) από το μέσο της προσομοίωσης σε μεγαλύτερη κλίμακα.



Figure 2: Λεπτομερής απεικόνιση τριών συνεχόμενων καρέ (11, 12, 13) από το Demo 1, αναδεικνύοντας τη διαδοχική περιστροφή.

- **Ομαλότητα και Φορά Κίνησης:** Το αντικείμενο εκτελεί μια πλήρη και απόλυτα ομαλή περιστροφή προς τη σωστή δεξιόστροφη (clockwise) κατεύθυνση, επιβεβαιώνοντας αφενός τον ορθό μαθηματικό υπολογισμό του πίνακα περιστροφής και αφετέρου την επιτυχή ενσωμάτωση της επιθυμητής φοράς στο εσωτερικό της κλάσης *Trafo*. Τα συνεχόμενα καρέ αποδεικνύουν τη μικρή, σταδιακή μετατόπιση των κορυφών μεταξύ των frames.
- **Σταθερότητα Υφής:** Η εικόνα υφής παραμένει απόλυτα "προσκολλημένη" στη γεωμετρία χωρίς παραμορφώσεις ή ολίσθηση (texture swimming). Αυτό αποδεικνύει ότι ο μετασχηματισμός των κορυφών συνεργάζεται άψογα με τις UV συντεταγμένες.
- **Προοπτική Ορθότητα:** Παρατηρείται ξεκάθαρα η χαρακτηριστική σμίκρυνση των τμημάτων του αντικειμένου που περιστρέφονται και απομακρύνονται από την κάμερα, επιβεβαιώνοντας τη μαθηματική ακρίβεια της διαίρεσης με το βάθος.

5.4.2 Αποτελέσματα Demo 2: Κυκλική Τροχιά Κάμερας

Αντίστοιχα, το παρακάτω πλέγμα παρουσιάζει την αλληλουχία της κίνησης όταν το αντικείμενο παραμένει απόλυτα σταθερό στο Παγκόσμιο Σύστημα Συντεταγμένων και η κάμερα διαγράφει τροχιά γύρω από αυτό.

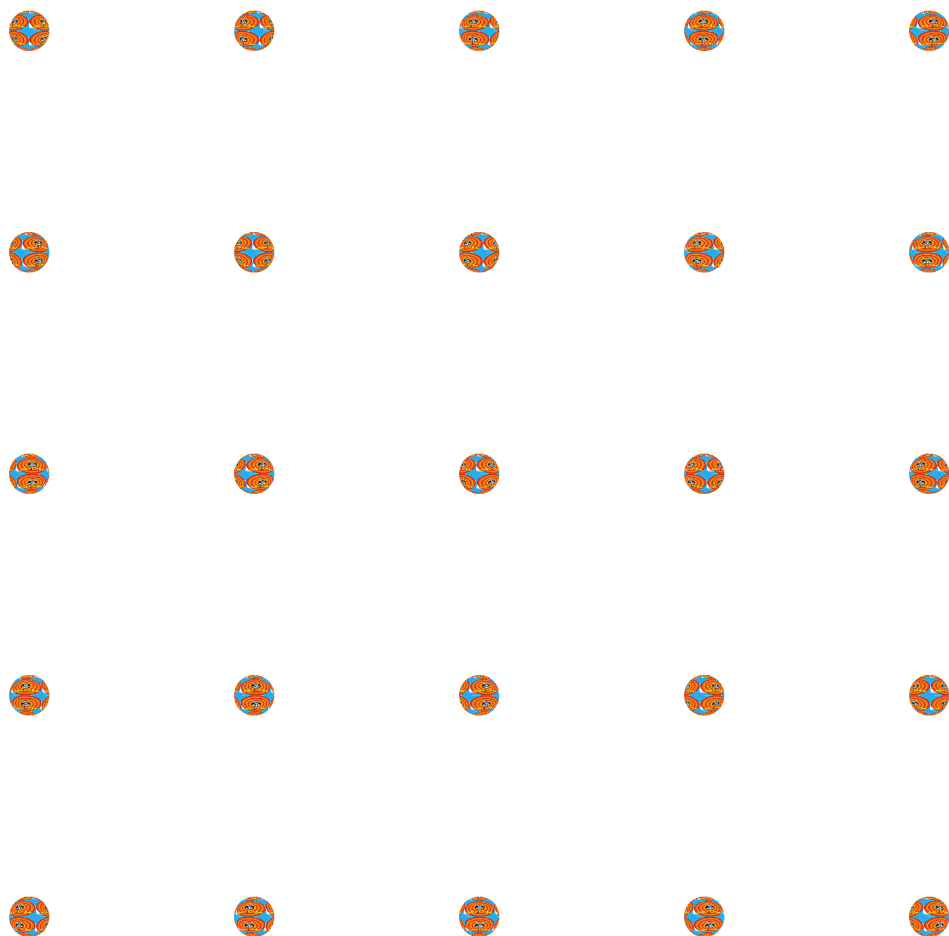


Figure 3: Σύνολο 25 καρέ από το Demo 2: Κυκλική τροχιά της κάμερας γύρω από το σταθερό αντικείμενο.

Ακολουθούν 3 συνεχόμενα καρέ (11, 12 και 13) σε μεγαλύτερη κλίμακα για τη λεπτομερή παρατήρηση της προοπτικής αλλοίωσης και της επικάλυψης των επιφανειών κατά τη μετατόπιση της κάμερας.



Figure 4: Λεπτομερής απεικόνιση τριών συνεχόμενων καρέ (11, 12, 13) από το Demo 2.

- **Απόσταση Κάμερας-Αντικειμένου:** Παρατηρείται ότι το αντικείμενο εμφανίζεται αισθητά πιο μικρό και απομακρυσμένο σε σχέση με το Demo 1. Αυτό οφείλεται στην παράμετρο `radius (k_cam_radius)` που καθορίζει την ακτίνα της τροχιάς της κάμερας στον χώρο. Η τιμή της ακτίνας είναι αρκετά μεγάλη ώστε να διασφαλίζεται ότι το αντικείμενο θα παραμένει πλήρως ορατό μέσα στο οπτικό πεδίο της κάμερας από οποιαδήποτε γωνία λήψης, αποτρέποντας φαινόμενα αποκοπής (clipping) στα άκρα της εικόνας.
- **Συνεχής Εστίαση:** Παρόλο που η κάμερα κινείται διαρκώς, το αντικείμενο παραμένει αυστηρά κεντραρισμένο. Αυτό αποδεικνύει την απόλυτη ορθότητα της συνάρτησης `lookat` και τον σωστό επαναυπολογισμό της ορθοκανονικής βάσης της κάμερας σε κάθε καρέ.
- **Σχετικότητα Κίνησης και Φορά Τροχιάς:** Ενώ το αντικείμενο είναι στατικό, η κυκλική τροχιά της κάμερας γύρω από αυτό δημιουργεί ένα οπτικό αποτέλεσμα ισοδύναμο με την περιστροφή του αντικειμένου. Μέσα από τα διαδοχικά καρέ επιβεβαιώνεται οπτικά η επιθυμητή δεξιόστροφη (clockwise) κίνηση της κάμερας (λόγω της αρνητικής γωνίας), η οποία παράγει μια φαινομενικά αντίρροπη περιστροφή του αντικειμένου. Αυτό επικυρώνει τη βασική αρχή των γραφικών υπολογιστών ότι ο μετασχηματισμός View Matrix είναι πρακτικά ο αντίστροφος του Model Matrix.
- **Επίλυση Κρυφών Επιφανειών:** Στα καρέ υψηλής ανάλυσης διαπιστώνεται πως δεν υπάρχουν γεωμετρικά σφάλματα επικάλυψης. Τα πολύγωνα που βρίσκονται πιο κοντά στην κάμερα κρύβουν σωστά τα πιο απομακρυσμένα, γεγονός που επαληθεύει τον ορθό υπολογισμό του βάθους (`depth`) και τη σωστή ταξινόμησή τους από τον Painter's Algorithm.